Féidearthachtaí as Cuimse
Infinite Possibilities

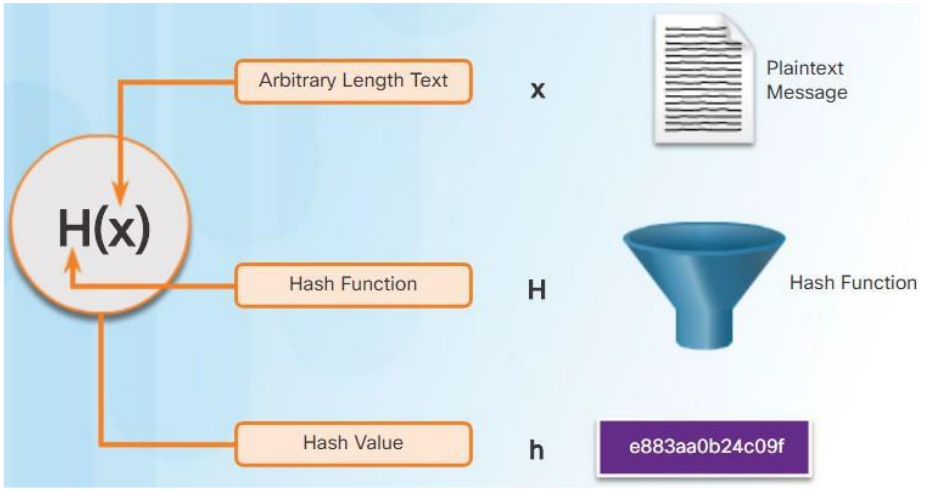
Secure Communications & Cryptography (MACS H6014)

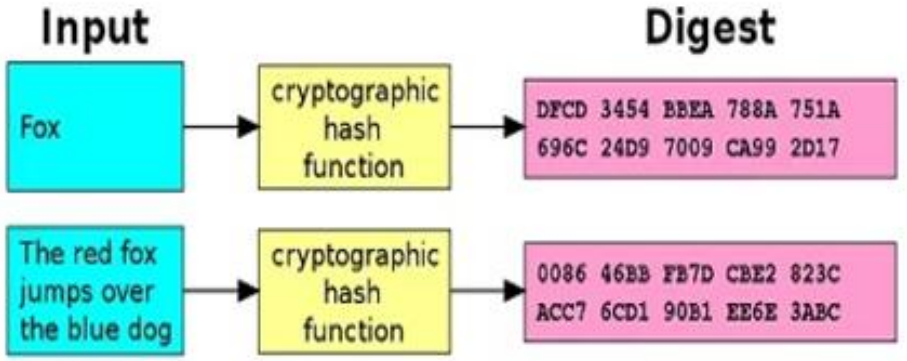
Cryptographic Hashing

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Cryptographic Hash Function

- A cryptographic hash function takes an arbitrary data block as input and returns a fixed-size bit string as output. The returned value is called the ***cryptographic hash value***.
- The cryptographic hash function creates a ***hash value*** so that any (accidental or intentional) change to the data will change the hash value.
- Therefore, it is extremely unlikely that some other text will produce the same hash value.
- The data to be encoded are often called the message, and the ***hash value*** is sometimes called the ***message digest*** or ***simply digest***.
- In the cryptographic hash function, even small changes in the input would cause a large change in the output.
- The figure shows how the slight changes in input (here in the word "over") drastically change the resulting output.

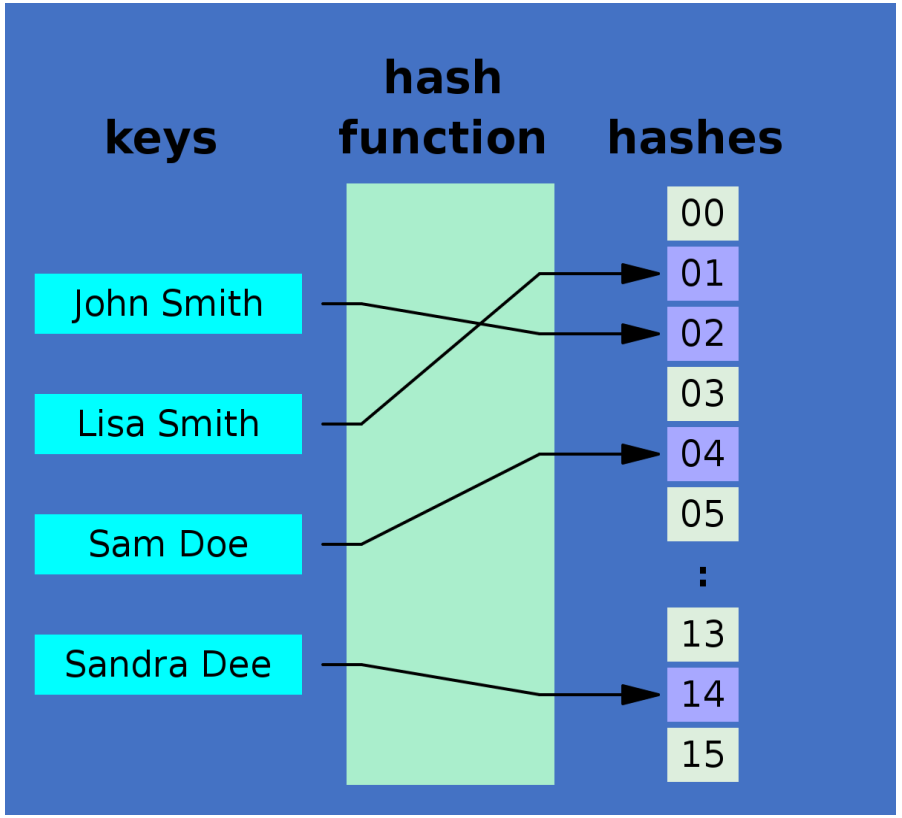




muhammad.arshad@tudublin.ie

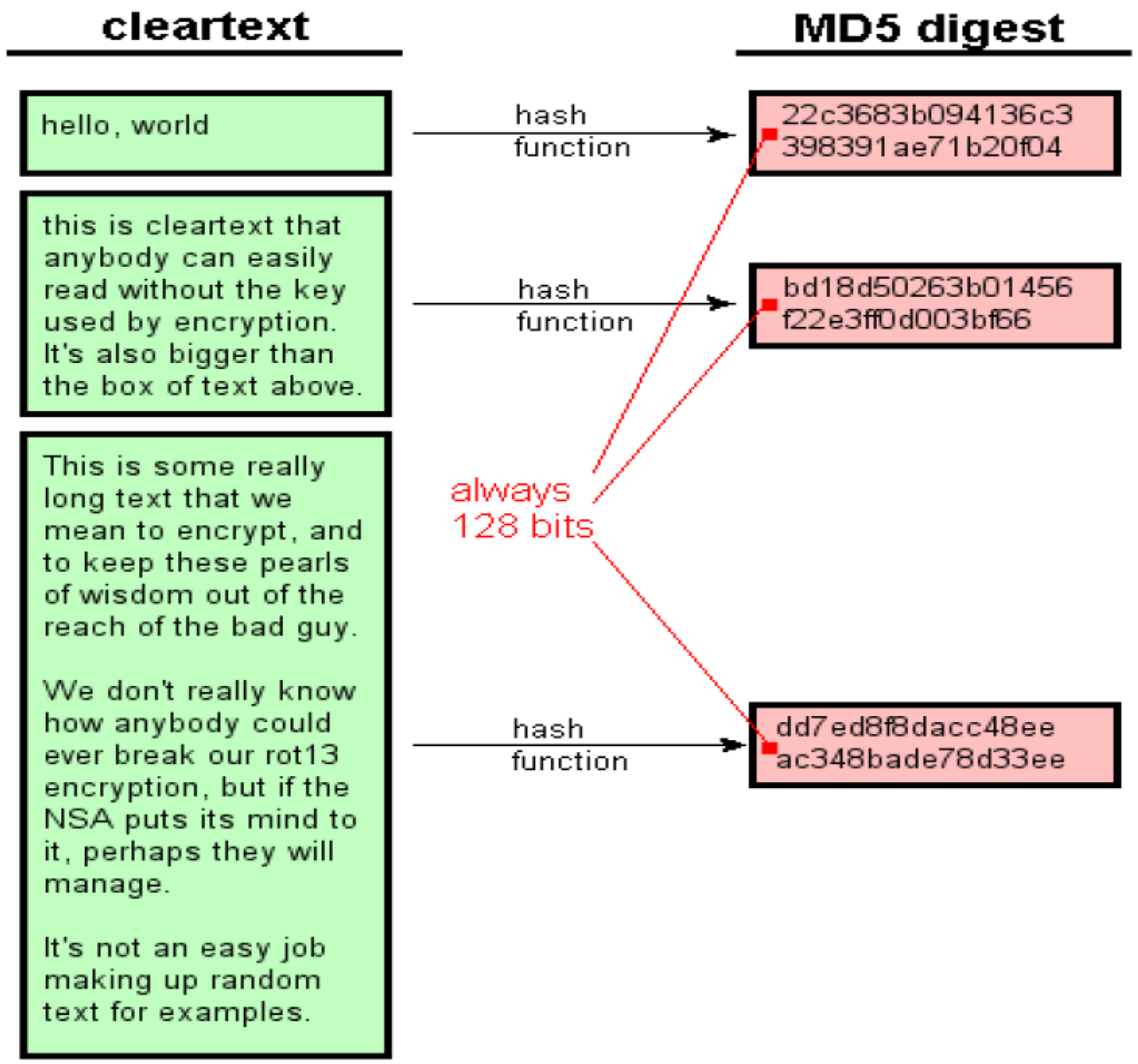
Hash Function Working

- A hash function is a formula or an algorithm that-
  - takes large data sets of variable length as input, and
  - returns smaller data sets of fixed length as output.
- Since, the output is smaller than the input data, a hash function compresses an ***n***-bit message string to create an ***m***-bit string where ***n*** is normally greater than ***m***.
- The values returned by a hash function are called *hash values, hash codes, hash sums, checksums, or simply hashes*.
- The *hash function* creates a hash value in such a way that it is extremely unlikely that some other text will produce the same hash value.
- A ***hash table*** (also called ***hash map***) is used to implement an associative array that can map keys to values.
- A hash table uses a hash function to compute an index into an array of buckets or slots, from which the correct value can be found. So you might get something like:
  - 01 Lisa Smith
  - 02 John Smith
  - 04 Sam Joe
  - 14 Sandre Dee



muhammad.arshad@tudublin.ie

Hashing (Example)



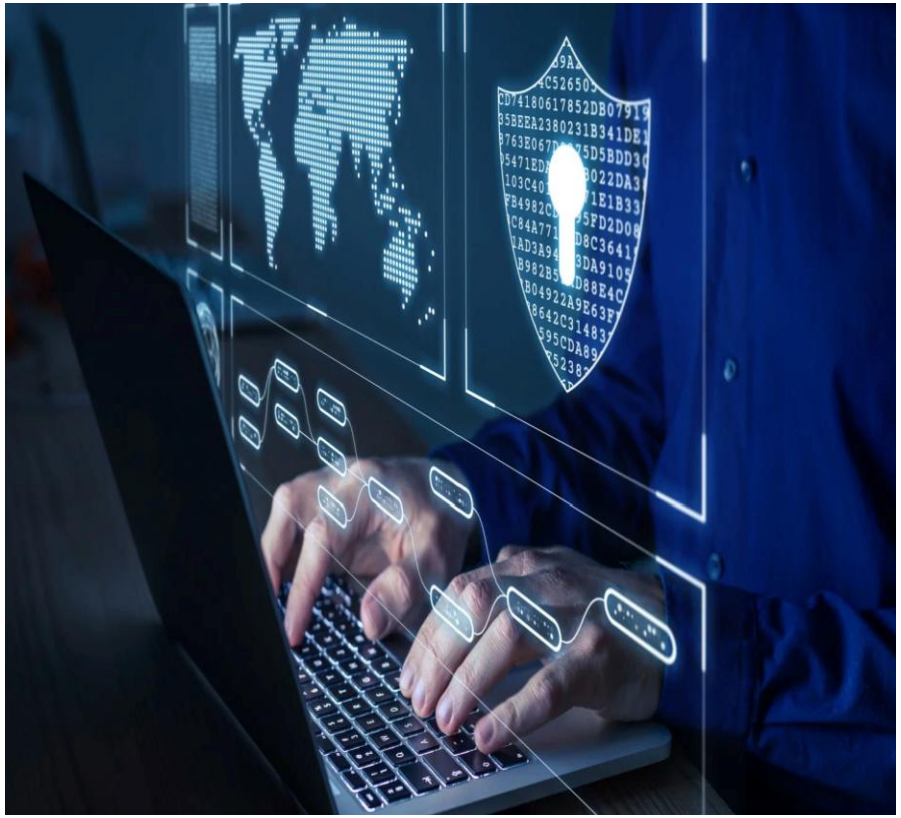
- Hashes compile a stream of data into a small digest (a summarized form) and it's strictly a **one-way operation**.
- All hashes of the same type in this example show the "MD5" variety has the same output size no matter how big the inputs are:

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Importance of Hashing

- Hashing is used to index and retrieve items in a database because it is faster to find the item using the shorter hashed key.
- In addition to faster data retrieval, hashing is also used to encrypt and decrypt digital signatures (used to authenticate message senders and receivers).
- Hashing plays a vital role in security systems where it is used to ensure that transmitted messages have not been tampered with.
- The sender generates a hash of the message, encrypts it, and sends it with the message itself.
- The recipient then decrypts both the message and the hash, produces another hash from the received message, and compares the two hashes.
- If they are the same, there is a very high probability that the message was transmitted intact.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TUDUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Applications of Hash Function

- Cryptographic hash functions have many information security applications, such as:
  - digital signatures
  - message authentication codes (MACS)
  - Other forms of authentication
- Hash functions ***are primarily used*** to generate fixed-length output data that acts as a shortened reference to the original data.
- This is useful when the output data is too cumbersome to use in its entirety.
  - ***For example***, consider a list of person's names. Here, the name of each person is of variable length. Searching for a person's name in the list is slow; the time required to retrieve each name may also vary. But if each name could be hashed to a fixed length integer, then searching and retrieving each name will be performed in faster with constant time.
- Hash functions are also used to accelerate table lookup or data comparison tasks such as *finding items in a database, detecting duplicated or similar records in a large file, finding similar stretches in DNA sequences, and so on*.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Applications of Hash Function in Cryptography

- Hash functions are used for:
  - Verifying the integrity of message and file
  - Verifying password for secure login
  - Fingerprints of keys
  - Authentication
  - Digital signatures
- **Verifying the integrity of files or messages**
  - An important application of secure hashes is the verification of message integrity.
    - *For example*, determining whether any changes have been made to a message (or a file) can be accomplished by comparing message digests calculated before and after transmission (or any other event).
  - For this reason, most digital signature algorithms only confirm the authenticity of a hashed digest of the message to be "signed".
  - Verifying the authenticity of a hashed digest of the message is considered proof that the message itself is authentic.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Applications of Hash Function in Cryptography (Contd.)

➢ Verifying password for secure login

- ❖ A related application of the hash function is password verification.
- ❖ Storing all user passwords as plaintext characters can result in a massive security breach if the password file is compromised.
- ❖ One way to reduce this danger is to only store the hash digest of each password instead of the plaintext password in the table (a file) that is stored by user identification.
- ❖ Any user can read the contents of the file, but, because the hash function is a one-way function, it is almost impossible to guess the value of the password.
- ❖ When the password is created, the system hashes it and stores the hash in the password file.
- ❖ When the user sends her user ID and password, the system creates a hash of the password and then compares the hash value with the one stored in the file.
- ❖ If there is a match, the user is granted access; otherwise, access is denied.



➢ Fingerprints of keys

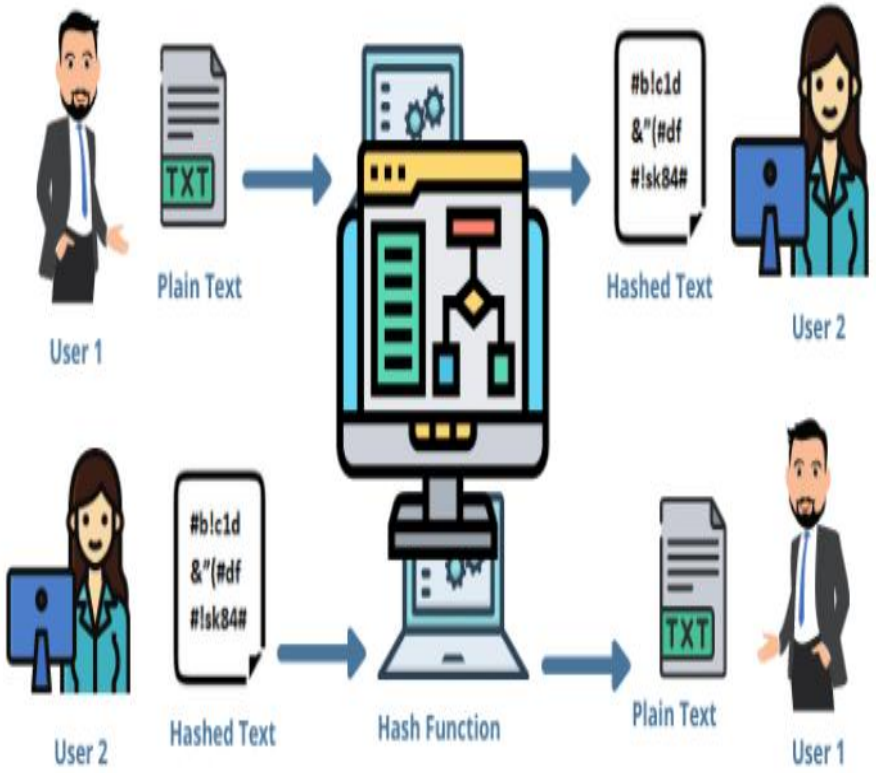
- ❖ A message digest can also serve as a means of reliably identifying a file.
- ❖ One of the main applications of a hash function is to allow the fast look-up of data in a hash table.
- ❖ Being hash functions of a particular kind, cryptographic hash functions lend themselves well to this application too.

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Applications of Hash Function in Cryptography (Contd.)

- **Authentication**
  - Authentication is the assurance that the communicating entity is the one that it claims to be.
  - A cryptographic hash function can be used to provide authentication.
- **Digital Signature**
  - A digital signature, first proposed in 1976 by Whitfield Diffie of Stanford University, is a digital code (encrypted message digest).
  - It can be attached to an electronically transmitted message that uniquely identifies the sender.
  - Like a written signature, the purpose of a digital signature is to guarantee that the individual sending the message really is who he or she claims to be.
  - It is linked to the data in such a manner that if the data is changed, the digital signature is invalidated.
  - When making a digital signature, cryptographic hash functions are generally used to construct the message digest.
  - A digital signature serves three important purposes:
    - Verifies data integrity.
    - Provides authentication of the sender.
    - Provides non-repudiation.

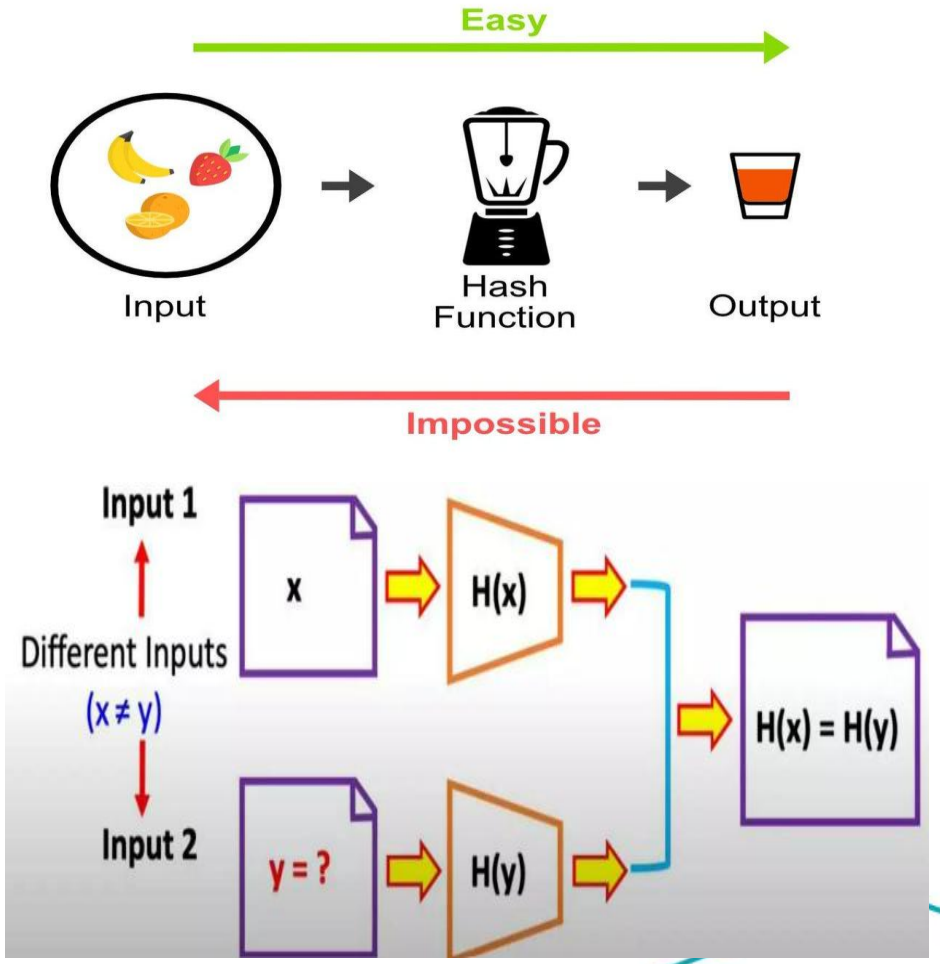


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Properties of Cryptographic Hash Function

- A cryptographic hash function must be able to withstand all known types of cryptanalytic attacks.
- A desirable cryptographic hash function should have the following properties:
  - A hash function produces a fixed length value from a variable length source.
  - It is easy to compute the hash value for any given message.
- **Pre-image resistance:** Given a hash ***h***, it should be difficult to find any message ***m*** such that h = hash(m). That is, it is infeasible to generate a message that has a given hash.
- A function with this property is called a one-way function.
- Functions that lack this property are vulnerable to preimage attacks.
- **Second pre-image resistance:** Given a message m_1, it should be difficult to find another message m_2 such that $m_1 \neq m_2$, that hashes to the same value, i.e. $hash(m_1) = hash(m_2)$.
- Functions that lack this property are vulnerable to second pre-image attacks.
- It is infeasible to modify a message without changing the hash.

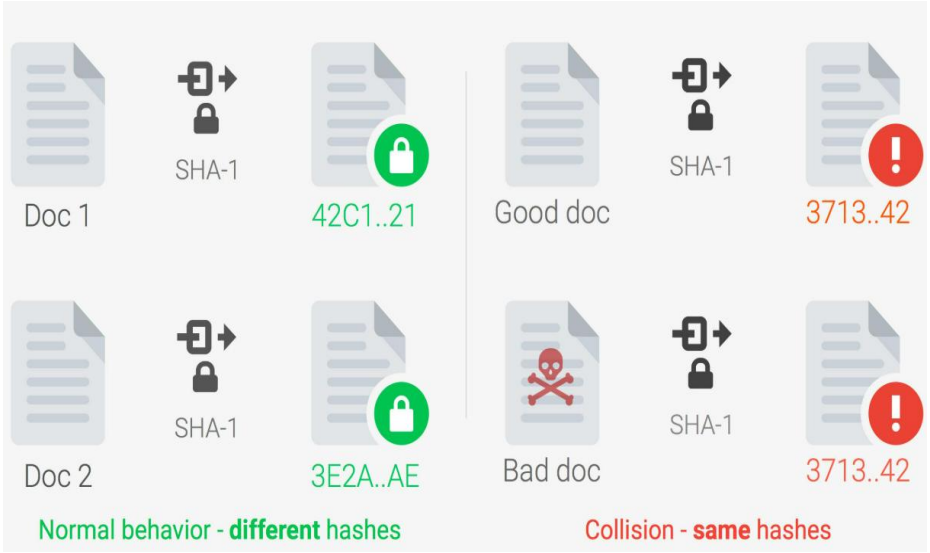


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Properties of Cryptographic Hash Function (Contd.)

- **Collision resistance:** It should be difficult to find two different messages m_1 and m_2 with the same hash h. Such a pair is called a cryptographic hash collision. This property is sometimes referred to as strong collision resistance.
- A hash function must be referentially transparent, i.e., if called twice on input that is "equal" (for example, strings that consist of the same sequence of characters), it should give the same result.
- A hash procedure must be deterministic—meaning that for a given input value, it must always generate the same hash value.
- Hash functions are destructive, as the original data is lost when hashed.
- A small change in the input m would cause a large change in the output of the hash function.
- It should be impossible for an adversary to find two messages with substantially similar digests; or to infer any useful information about the data, given only its digest.
- Therefore, a cryptographic hash function should behave as much as possible like a random function while still being deterministic and efficiently computable.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Division-Remainder Method

- Using this method, choose a number ***m*** that is larger than the number ***n*** of keys in ***K*** (***K*** is a set of keys). Generally, the number ***m*** is chosen to be a prime number. The hash function **H** is defined as:

  k (mod m) or, *k (mod m)+1*

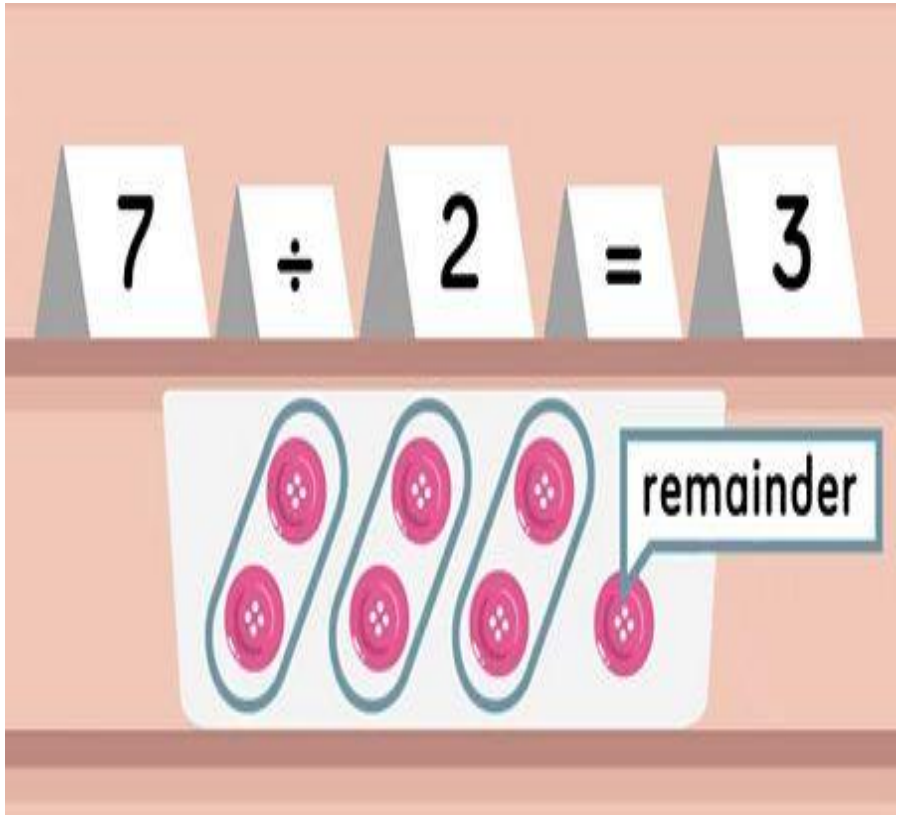
- Here k (mod m) denotes the remainder when ***k*** is divided by ***m***.
- The second formula is used when we want the hash addresses to range from **1 to m** rather than from **0 to m-l**.

Example: Suppose a company with 68 employees assigned a 4-digit employee number to each employee which is used as the primary key. Apply the division method of the hash function to each of the following employee numbers: 3205, 7148, 2345.

Solution:

- Since there are 68 employees in the company, a two-digit employee number is sufficient to represent them.
- The highest 2-digit number is 99 and 97 is the nearest 2-digit prime number of 99. So, we divide each of the 4-digit employee numbers by 97.

  H(3205) = 3205 (mod 97) = 04;
  H(7148) = 7148 (mod 97) = 67;
  H(2345) = 2345 (mod 97) = 17;
- In the case that the memory addresses begin with 01 rather than 00, we choose the function *H(k) = k(mod m)+1* to obtain. H(3205)= 3205 (mod 97)+1 = 4+1=05.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Mid-Square Method

- Using this method, the key ***k*** is squared. Then the hash function ***H*** is defined by:

  $H(k) = I$,

  Where ***I*** is obtained by deleting digits from both ends of k^2.

Example: Suppose a company with 68 employees assigned a 4-digit employee number to each employee which is used as the primary key. Apply the mid-square method of the hash function to each of the following employee numbers: 3205, 7148, 2345.

Solution:

- Since there are 68 employees in the company, a two-digit employee number is sufficient to represent them.

K	3205	7148	2345
K^2	10272025	51093904	5499025
H(k) = I	72	93	99

- Observe that the 4th and 5th digits counting from right are chosen for the hash address.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Folding Method

- Using this method, the key ***k*** is portioned into a number of parts, k_1, k_2, k_3 ..... k_r, where each part, except possibly the last, has the same number of digits as the required address. Then the parts are added together, ignoring the last carry if any. That is:
- $H(k) = k_1+k_2+k_3+.....k_r$ where the leading-digit carries, if any, are ignored.

Example: Suppose a company with 68 employees assigned a 4-digit employee number to each employee which is used as the primary key. Apply the folding method of the hash function to each of the following employee numbers: 3205, 7148, 2345.

Solution:

- Since there are 68 employees in the company, a two-digit employee number is sufficient to represent them.
- Chop the key into two parts and then add them.

K	3205	7148	2345
$H(k)=K_1+k_2$	32+05=37	71+48=119	23+45= **68**
H(k)	37	19	**68**

- Observed that the leading digit from the 2nd function is ignored.
- Alternatively, the reverse folding method may be used as:

  H(3205) = 32+50 = 82

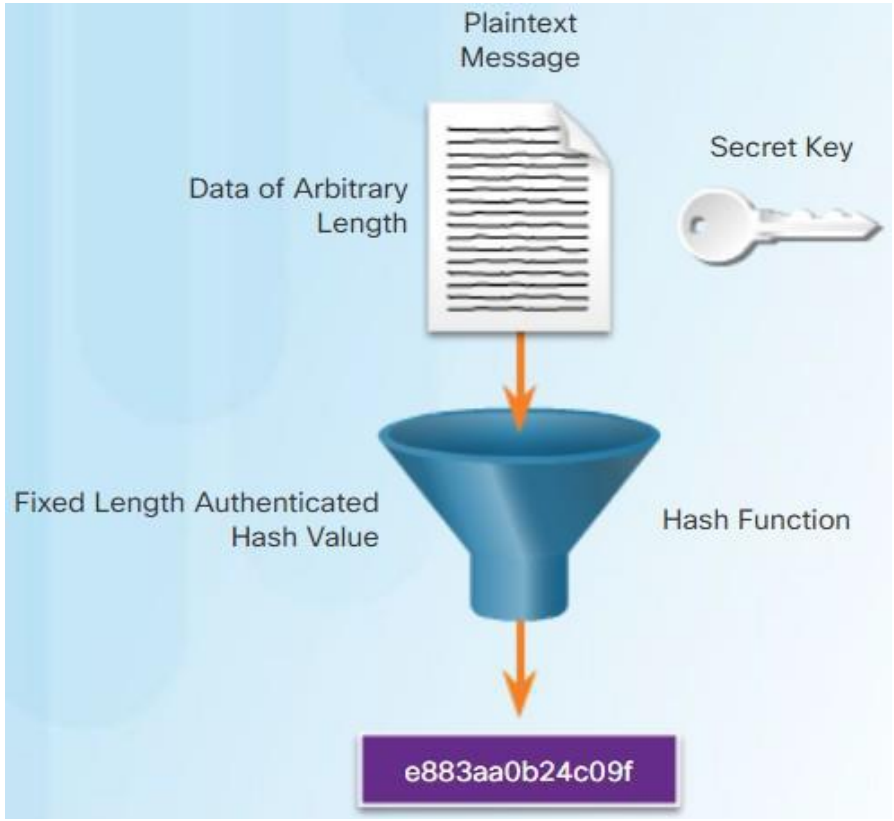


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Keyed Hash Functions

- Also known as **Hash-based Message Authentication Codes (HMAC)**.
- Provide a way to verify the integrity and authenticity of data.
- Keyed hash functions combine a cryptographic hash function with a secret key to produce a fixed-size hash value (digest) of the input data.
- The keyed hash function, or HMAC, typically works:
  - **Input Data:** You have some data that you want to protect, often referred to as the message or the plaintext.
  - **Secret Key:** You and the recipient share a secret key. This key is kept confidential and should not be disclosed to unauthorized parties.
  - **Hash Function:** A cryptographic hash function is used to process the combination of the secret key and the input data. This operation results in a hash value or digest.
  - **Hash Output:** The hash value, or HMAC, is sent along with the original data to the recipient.
  - **Verification:** The recipient, who also has the secret key, repeats the same process on the received data and key to generate a hash value.
- Common cryptographic hash functions used in HMAC constructions include SHA-256, SHA-384, and SHA-512.

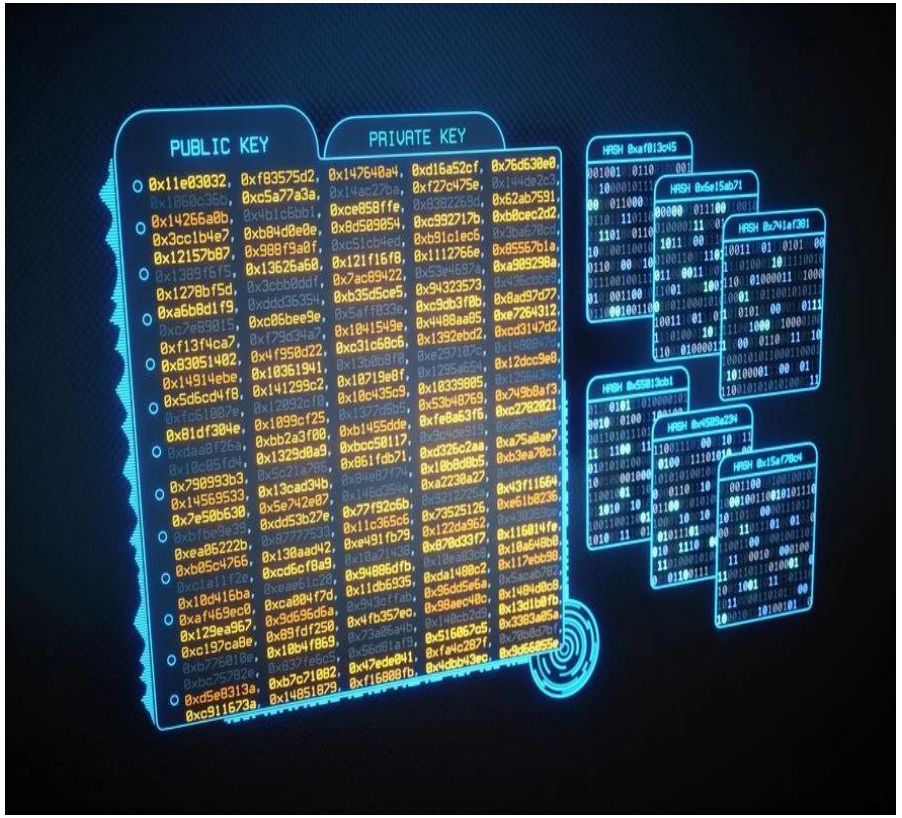


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Unkeyed Hash Functions

- Also known as **Cryptographic Hash Functions** or **One-way Hash Functions**.
- Mathematical functions that take an input (or message) and produce a fixed-size string of characters, typically a hexadecimal string of digits.
- Keyed hash functions combine a cryptographic hash function with a secret key to produce a fixed-size hash value (digest) of the input data.
- Several important characteristics are:
  - **Deterministic:** Given the same input, an unkeyed hash function will always produce the same output.
  - **Fast Computation:** Hash functions are designed to be computationally efficient and produce their output quickly.
  - **Pre-image Resistance:** It should be computationally infeasible to determine the original input from its hash value.
  - **Collision Resistance:** It should be difficult to find two different inputs that produce the same hash value.
  - **Avalanche Effect:** A small change in the input should result in a significantly different hash output.
  - **Fixed Output Length:** Hash functions produce a fixed-size output, regardless of the size of the input.

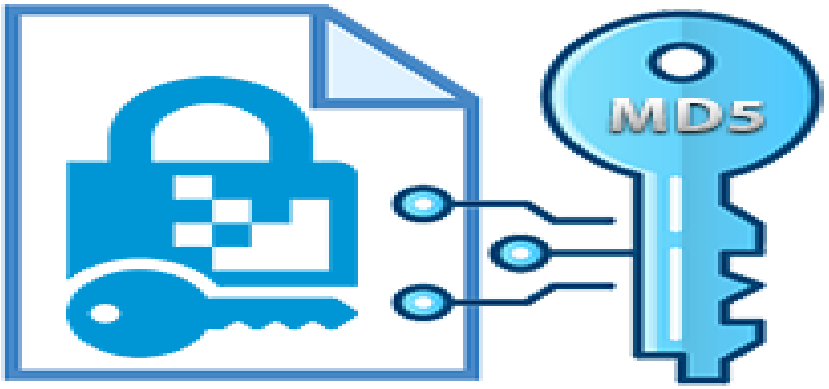


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Hash Function Used in Cryptography

- The two commonly used hash functions are MD and SHA.
  - **MD:**
    - MD stands for Message Digest.
    - Several MD hash algorithms designed by Ron Rivest are MD2, MD4, and MD5.
    - The last version MD5 is more secure than the previous versions.
    - It divides the message into blocks of 512 bits and creates a 128-bit digest.
  - **SHA:**
    - SHA stands for Secure Hash Algorithm.
    - This standard was developed by NIST (National Institute of Standards and Technology).
    - This standard is mostly based on MD5.
    - Several versions of the SHA standard were released: SHA-I, SHA-3, SHA-224, SHA-256, SHA-384 and SHA-512.
    - SHA returns a string of 160 bits.
    - Both MD5 and SHA-I hash functions are built with the **Merkle-Damgard** construction.



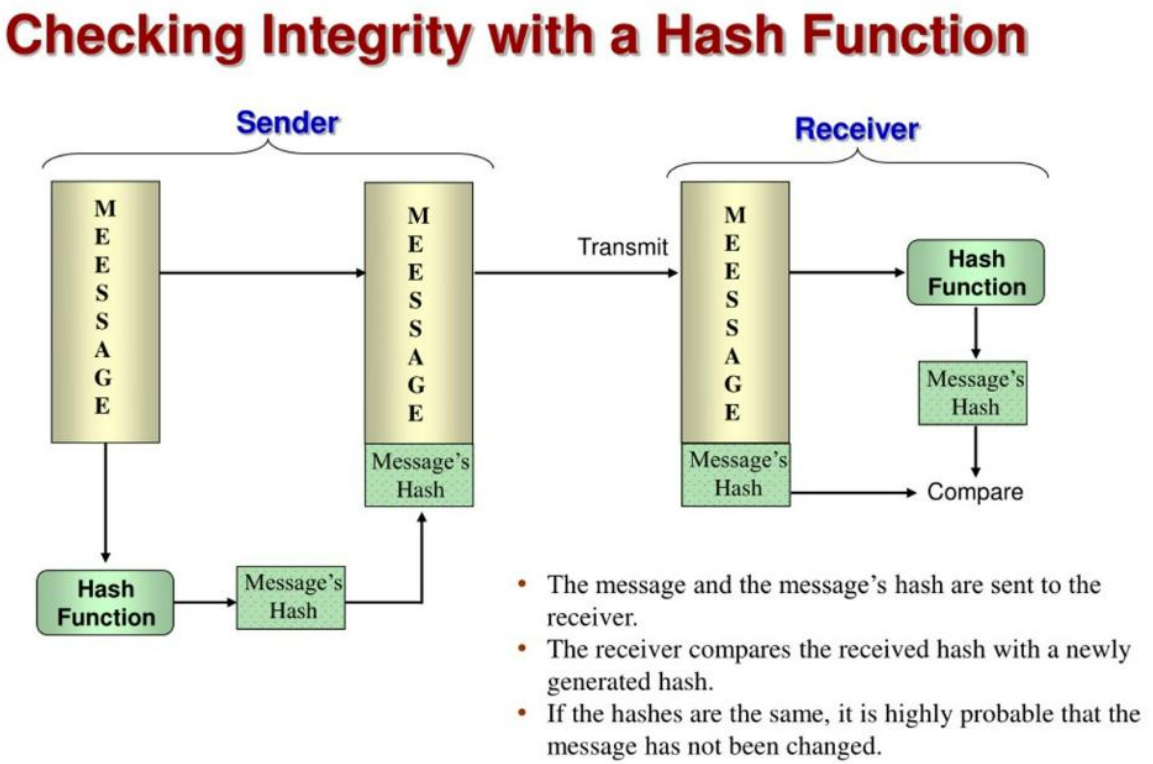


muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

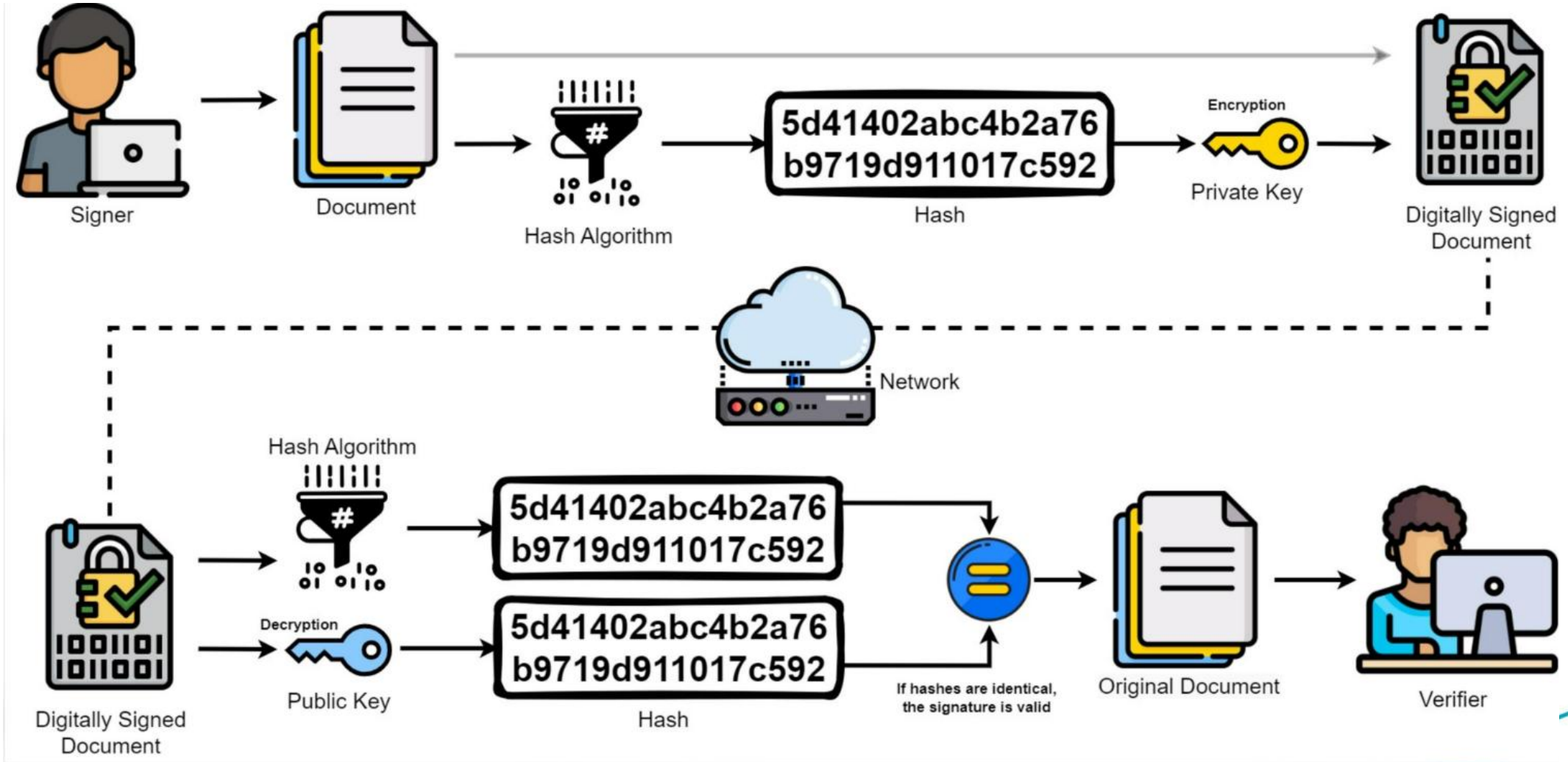
Real-World Example: Verifying a University Examination Paper

- The examination board prepares the original file.
- A SHA-256 hash function processes the file.
- The system produces a 256-bit message digest.
- The file and its expected digest are distributed securely.
- The examination center calculates the digest of the received file.
- The center compares the calculated digest with the expected digest.
- Matching digests indicate that the file is consistent with the expected version; differing digests indicate that it is not.



muhammad.arshad@tudublin.ie

Message Digest in Digital Signatures

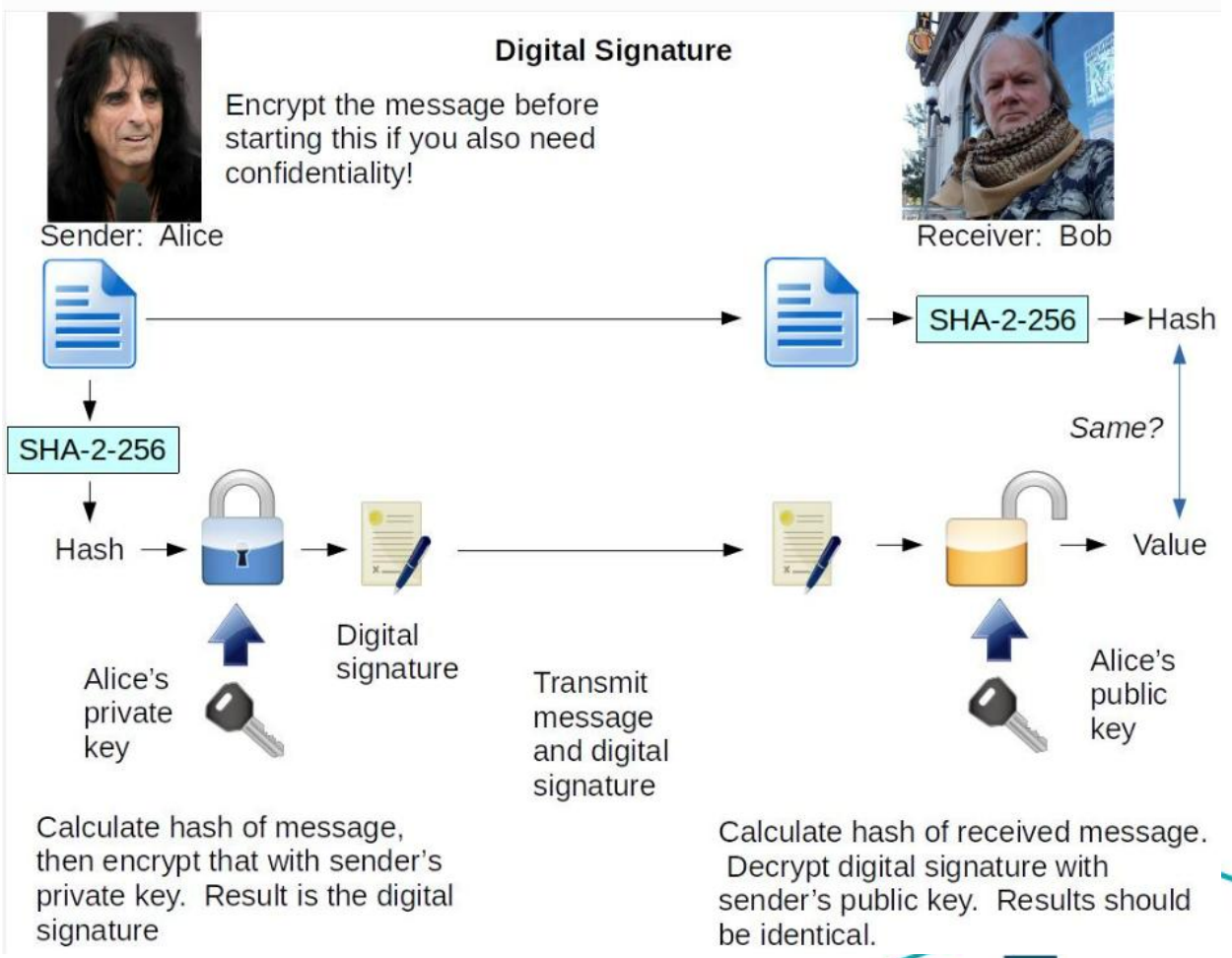


muhammad.arshad@tudublin.ie

Message Digest in Digital Signatures (Contd.)

Digital Signatures help provide:

- **Integrity:** Detecting changes to signed data.
- **Authentication:** Verifying that a signature was produced using the corresponding private key, subject to correct key management.
- **Support for non-repudiation:** Providing evidence that can support attribution, subject to legal, operational, and key-management considerations..



muhammad.arshad@tudublin.ie

Practical Comparison: MD5, SHA-1, and SHA-256

Algorithm	Digest size	Current security status	Recommended use
MD5	128 bits	Cryptographically broken for collision resistance	Not for security-sensitive integrity or signatures
SHA-1	160 bits	Collision resistance is broken	Avoid for new security-sensitive applications
SHA-256	256 bits	Widely used and currently considered secure for general cryptographic hashing	General-purpose integrity verification and cryptographic applications
SHA-3-256	256 bits	Modern cryptographic hash function based on a different construction from SHA-2	Suitable alternative for applications requiring a secure hash
MD5	128 bits	Cryptographically broken for collision resistance	Not for security-sensitive integrity or signatures

muhammad.arshad@tudublin.ie

NEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Sample Hash Formats

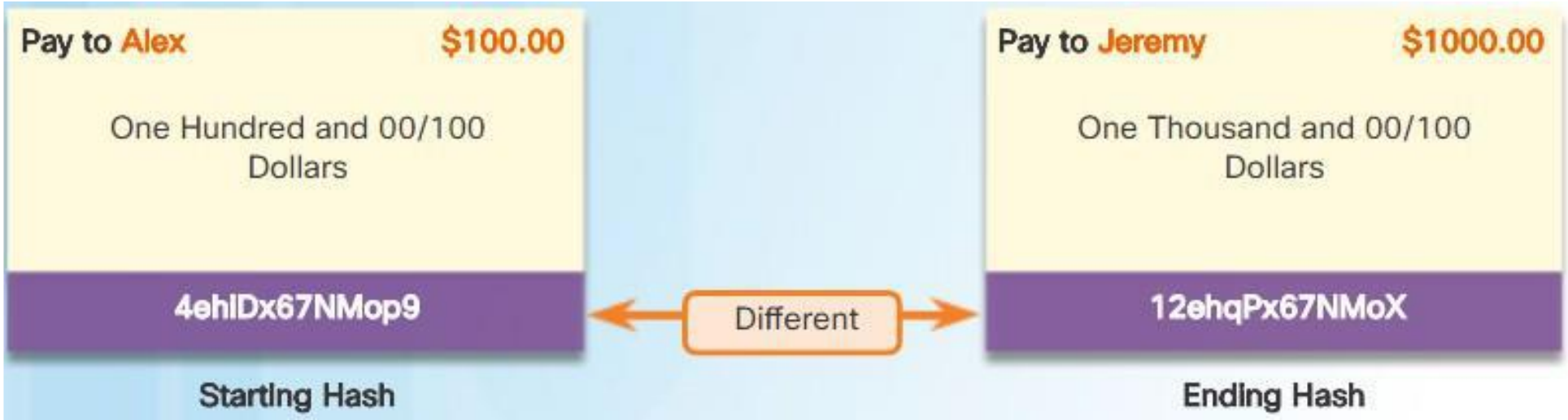
General Hashes	
MD5	5f4dcc3b5aa765d61d8327deb882cf99
SHA1	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
SHA256	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
SHA512	b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec049b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86
UNIX Hashes (with salt)	
DES	ZD3yxA4N/XZVg
MD5	1ZDzPE45C$EEQHJaCXI6yInV3FnskmF1
Bcrypt	$2a$12$111111111111111111111uAQxS9vJNRtBb6zeFDV6k7tyB0DZJF0a
MS Windows Hashes	
LM Hash	e52cac67419a9a224a3b108f3fa6cb6d
NT Hash	8846f7eaee8fb117ad06bdd830b7586c
MS DCC	c531cc9702cbbe9053dfa32d8940c2ca
MS DCC2	920873ab14cffe2420ebf69c6d5f8ee7
LDAP Hashes	
LDAP (MD5)	{MD5}X03MO1qnZdYdgyfeuILPmQ==
LDAP (MD5 Salted)	{SMD5}ZYxs6V7nZOz+ALwZu8nWglpEelBFNDVD
LDAP (SHA)	{SHA}W6ph5Mm5Pz8GgiULbPgzG37mj9g=
LDAP (SHA1 Salted)	{SSHA}Rr2ARpei2FyhmO51IpsE0S1np2BaRHpQRTQ1Qw==

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

MD5 and SHA

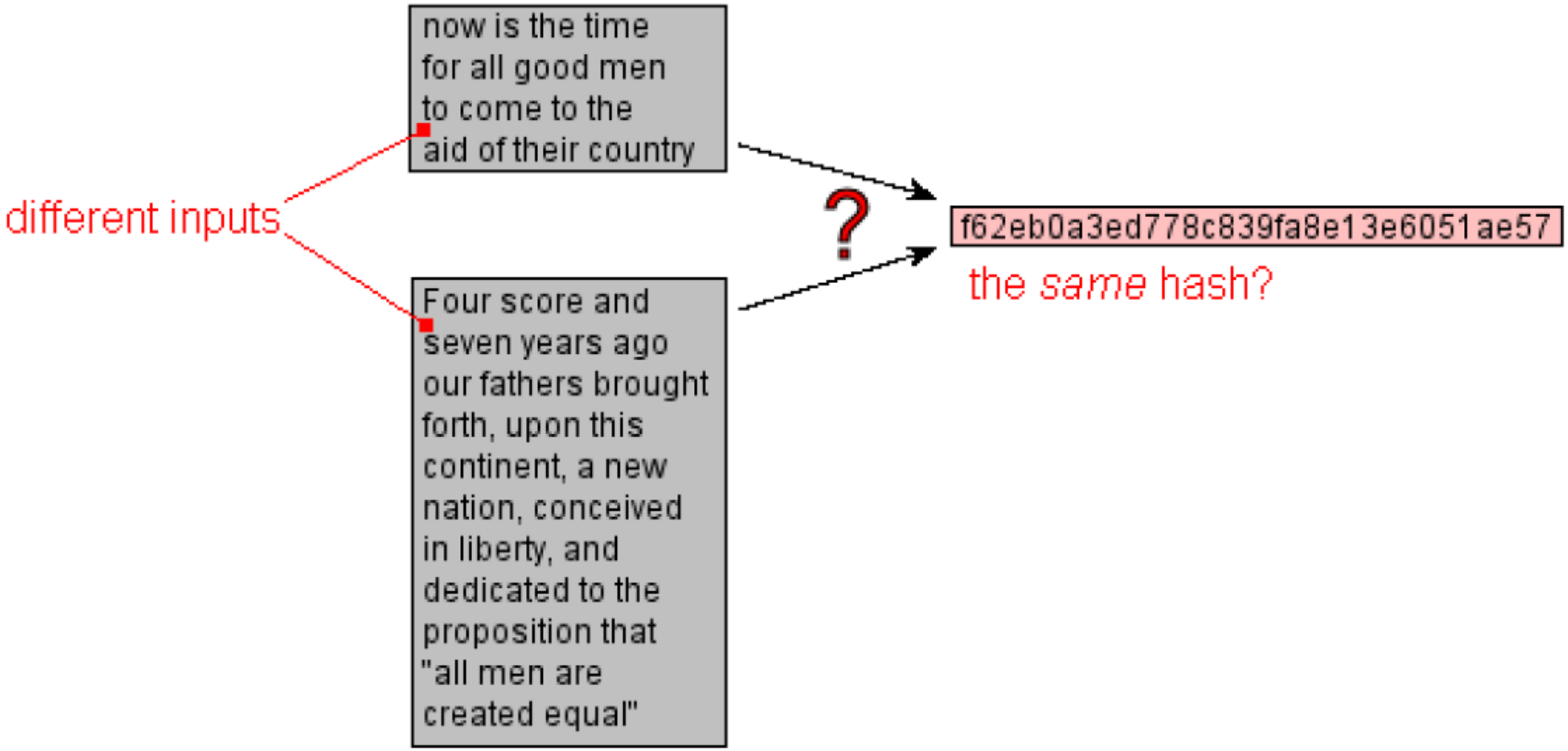
- Hash functions are used to ensure the integrity of a message. They ensure data has not changed in the message either accidentally or intentionally.
- While hashing can be used to **detect** accidental changes, it cannot be used to guard against deliberate changes. There is no unique identifying information from the sender in the hashing procedure.
- Vulnerable to **man-in-the-middle** attacks as anyone can create a hash for the data.



muhammad.arshad@tudublin.ie

What's a Hash Collison?

When different chunks of data produce the same hash value, this is known as a **collision**:



muhammad.arshad@tudublin.ie

What's a Hash Collison? (Contd.)

- A hash collision is when ***two different inputs give the same output***.
- Hash functions are designed to avoid this as best as they can.
- There are a fixed number of different output values for the hash function but you can give it any size input.
- 128-bit hash = 2^{128} 3.4 x 10^{38} possible values, which is:

  340,282,366,920,938,463,463,374,607,431,768,211,456 possible hashes
- *As there are more inputs than outputs, there is a probability some of the inputs will give the same output.*
- If the hash algorithm is properly designed and distributes the hashes uniformly over the output space, finding a hash collision by random guessing is ***highly unlikely***.
- Increasing the number of bits decreases the probability of finding a collision; for example, a 160-bit hash (SHA 1) has an output space of four billion times larger than that produced by MD5's 128 bits.
- MD5 and SHA1 have been attacked and made technically insecure due to engineering hash collisions.
- However, no attack has yet given a collision in both algorithms at the same time so if you use the MD5 hash AND the SHA1 hash to compare, you will see they're different.
- The MD5 collision example is available from https:// www.mscs.dal.ca/~selinger/md5collision/
- The SHA1 Collision is available from https://shattered.io/
- You shouldn't trust either algorithm for hashing passwords or data.

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Secure Hash Function 3

- Designed as a successor to the earlier SHA-1 and SHA-2 hash functions.
- Some key characteristics and features of SHA-3 include:
  - **Keccak Algorithm:** SHA-3 is based on the Keccak family and is known for its security properties, including resistance to known cryptographic attacks.
  - **Variable Output Length:** SHA-3 can produce hash values of different lengths, including 224 bits, 256 bits, 384 bits, and 512 bits, it is suitable for various applications.
  - **Resistance to Collision and Preimage Attacks:** SHA-3 is designed to be resistant to collision and preimage attacks, making it suitable for applications like data integrity verification and digital signatures.
  - **Security Analysis:** SHA-3 underwent a rigorous evaluation process, to ensure that the algorithm is secure against a wide range of potential attacks.
  - **Parallelism:** SHA-3 is designed to be easily parallelized, this allows for faster hash computation.
- SHA-3 is widely used in various cryptographic applications, including data integrity verification, digital signatures, and password hashing.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Secure Hash Function 256

- SHA-256, which is part of the SHA-2 family of hash functions.
- Here are some key characteristics and features of SHA-256:
  - **Fixed Output Length:** SHA-256 produces a fixed-size hash value of 256 bits (32 bytes). Applications are digital signatures, data integrity verification, and password hashing.
  - **Resistance to Collision and Preimage Attacks:** SHA-256 is designed to be resistant to collision attacks, and preimage attacks. These resistance properties are crucial for its security.
  - **High Speed:** SHA-256 is designed for high-speed implementation and is optimized for performance on modern computer architectures.
  - **Widely Used:** SHA-256 is widely adopted and used in many security protocols, including SSL/TLS for securing internet communications, cryptocurrency mining (e.g., Bitcoin), digital signatures, and more.
  - **NIST Standard:** SHA-256 is a standard defined by the NIST, and also includes SHA-224, SHA-256, SHA-384, and SHA-512, with varying output lengths.
  - **Security Analysis:** SHA-256 has undergone extensive security analysis, and It's considered secure and suitable for most cryptographic applications.



muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

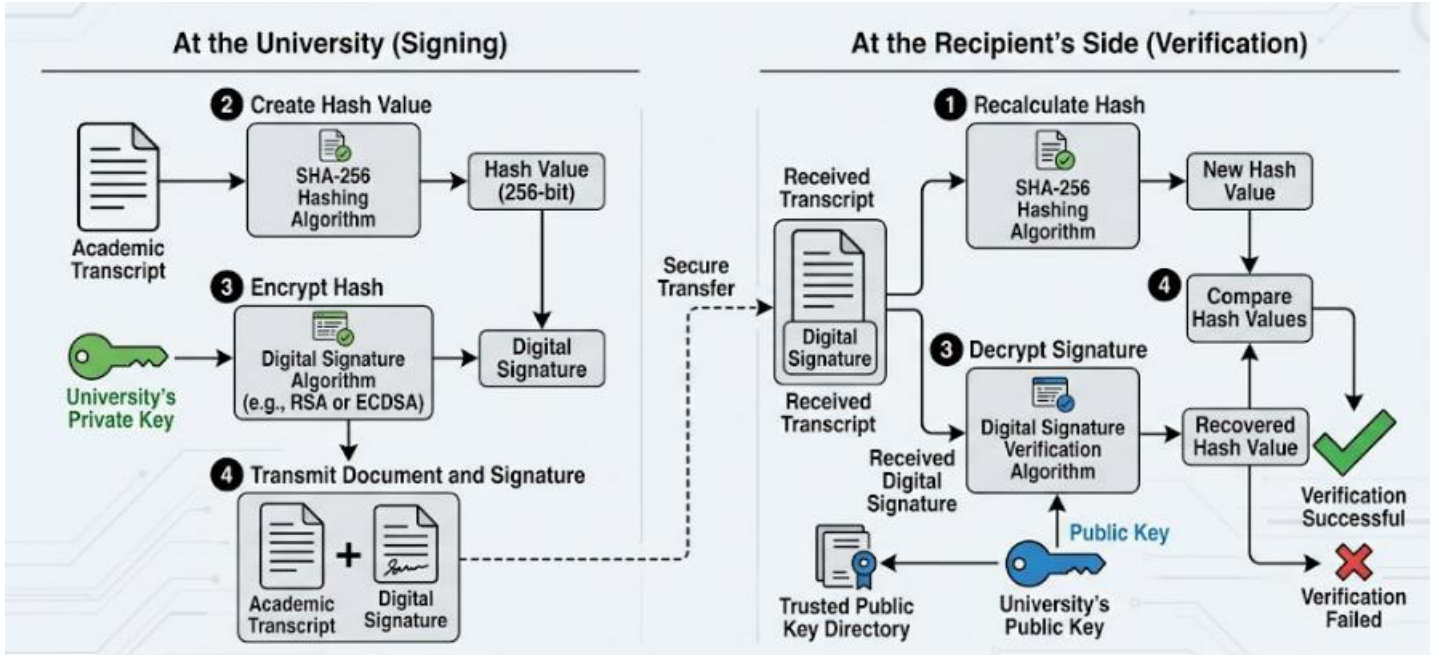
Advanced Application: Digitally Signed Academic Transcripts

At the university

- The university prepares the transcript.
- A digital-signature algorithm processes the document or its prescribed representation.
- The university signs the appropriate data using its private key.4. The transcript and signature are delivered to the recipient.

At the recipient's side

- The recipient receives the transcript and signature.
- The recipient obtains the university's public key from a trusted source.
- The recipient verifies the signature using the appropriate algorithm.
- If verification succeeds, the recipient has evidence that the signature is valid for the received document and corresponds to the relevant private key.



muhammad.arshad@tudublin.ie

Apache Hashing Methods

- The ***Apache-defined APR1*** format addresses the problems of brute forcing an MD5 hash, and iterates the hash value 1,000 times. This considerably slows an intruder as they try to crack the hashed value. The resulting hashed string contains ***"$apr1$"*** to identify the method and uses a 32-bit salt value. We can use ***htpassword*** on ***Openssl*** to compute the hashed string (where "Bill" is the user and "Hello" is the password).

```
anthony@Techs-MacBook-Air ~ % htpasswd -nbm bill hello
bill:$apr1$HFGqBAbB$HStRyveQjJ8.1XLOf/8Ux/

anthony@Techs-MacBook-Air ~ % openssl passwd -apr1 -salt PkWj6gM4 hello
$apr1$PkWj6gM4$XGWpADBVPyypjL/cL0XMc1
```

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Programming Hashing Methods (Contd.)

- **Passlib** is a password hashing library for **Python 2 and 3**, which provides cross-platform implementations of over 30 password hashing algorithms, as well as a framework for managing existing password hashes. It's designed to be useful for a wide range of tasks, from verifying a hash found in /etc/shadow, to providing full-strength password hashing for multi-user applications.

```
import passlib.hash;
salt="8sFt66rZ"
string="hello"
print ("APR1:"+passlib.hash.apr_md5_crypt.hash(string, salt=salt))
```

APR1:$apr1$PkWj6gM4$XGWpADBVPyypjL/cL0XMc1

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

PHPass Hashing Methods (Contd.)

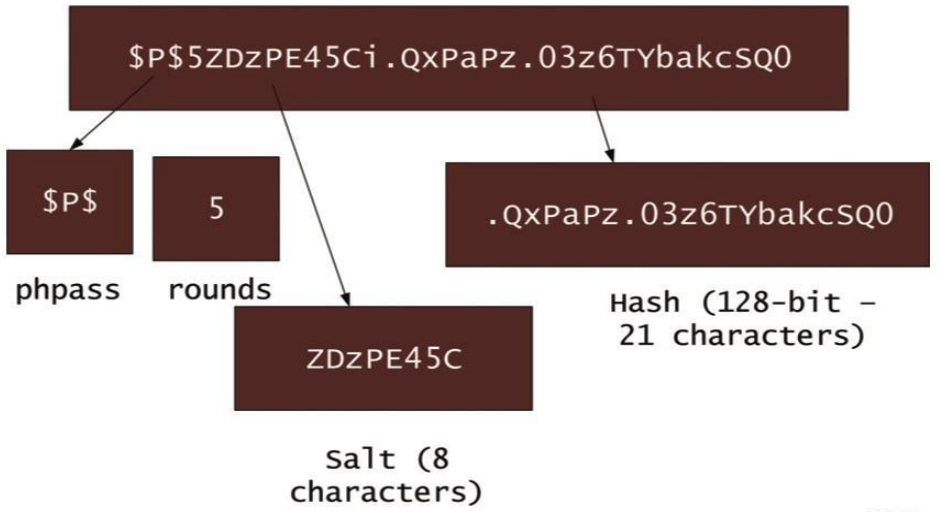
- Phpass is used as a hashing method by WordPress and is used within PHP applications. The three main methods used are:
- CRYPT BLOWFISH. This is the most secure method and relates to the Bcrypt hashing method.
- CRYPT EXT DES. This method uses DES encryption.
- MD5. This is the least preferred method and simply uses an MD5 digest.

The output uses the following to identify the differing types:

- "P". Standard
- H Phpbb.
- "1S"; Drupal with SHA 512 like digests.

A sample run with " and salt of " for seven rounds gives:

P5ZDzPE45Ci.QxPaPz.03z6TYbakcSQ0.



muhammad.arshad@tudublin.ie

Cisco Hashing Methods (Contd.)

- Much of the focus of protecting networks is around the server and host infrastructures.
- Intruders can often target simpler devices to gain a foothold, as these devices will typically have lower security levels and weaker passwords.
- This can include a range of Cisco networked devices that store passwords in a ***"Hashed"*** format, such as for Cisco phones, switches, and routers.
- An example configuration is:

```
hostname routerA
!
aaa new-model
aaa authentication login default local
aaa authentication ppp default if-needed local
enable secret 5 <hashvalue>
!
username fred password 7 <hashvalue>
username remote password 7 <hashvalue>
username guest password 7 <hashvalue>
```

muhammad.arshad@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

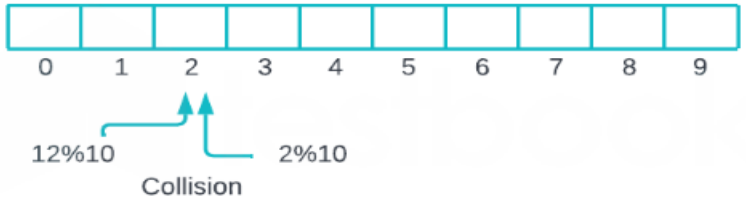
Short Quiz

- **Let us consider a list of numbers (34, 16, 2, 93, 80, 77, 51) and has table size is 10. What is the order of elements (from index 0 to size-1) in the has table?**
  1. null, null, 77, 16, null, 34, 93, 2, 51, 80.
  2. 77, 16, 34, 93, 2, 51, 80.
  3. 80, 51, 2, 93, 34, null, 16, 77, null, null. ✓
  4. 80, 51, 2, 93, 34, 16, 77.

The hash table is,

index	Value
0	80
1	51
2	2
3	93
4	34
5	
6	16
7	77
8	
9	

- **If the hash function is h(element) = element % 10, so for which of the pair of values, the collision will take place?**
  1. 15 and 14.
  2. 12 and 2. ✓
  3. 1 and 10.
  4. 11 and 22.



- **Which of the following searching techniques does not get affected by increasing the size of search list?**
  1. Binary search.
  2. Search by Hashing. ✓
  3. Linear search.
  4. None of the above.

- **Which of the following statement(s) is TRUE?**
  1. A hash function takes a message of arbitrary length and generates a fixed-length code.
  2. A hash function takes a message of fixed length and generates a code of variable length.
  3. A hash function may give the same hash value for distinct messages.

  1 only 2 and 3 only ✓ 1 and 3 only 2 only

muhammad.arshad@tudublin.ie

Practical Exercise

- **Week 6 – 23rd Oct**
  - Complete **Lab 7 – Hashing and Lab 14 – Introduction to Hashing** Worksheets
  - CryptoHack.org website - **Complete Hash Functions**
  - Read **Chapter 3 - Hashing** - Cryptography by William Buchanan
  - **Assignment 3** - Lab 8 - **Salted Hashes (Challenge Lab)** Due on 8th November

muhammad.arshad@tudublin.ie